

Remember that iterable objects are passed as an argument to the `iter()` function, whereas the object returned from `iter()` calls is an iterator object. Iterator objects are passed to the `next()` function. When you create your own data types with `class` statements, you can implement the `__iter__()` and `__next__()` special methods to use your objects in `for` loops.

Syntax vs. Runtime vs. Semantic Errors

There are many ways to categorize bugs. But at a high level you could divide programming errors into three types: syntax errors, runtime errors, and semantic errors.

Syntax is the set of rules for the valid instructions in a given programming language. A *syntax error*, such as a missing parenthesis, a period instead of a comma, or some other typo will immediately generate a `SyntaxError`. Syntax errors are also known as *parsing errors*, which occur when the Python interpreter can't parse the text of the source code into valid instructions. In English, this error would be the equivalent of having incorrect grammar or a string of nonsense words like, "by uncontaminated cheese certainly it's." Computers require specific instructions and can't read the programmer's mind to determine what the program should do, so a program with a syntax error won't even run.

A *runtime error* is when a running program fails to perform some task, such as trying to open a file that doesn't exist or dividing a number by zero. In English, a runtime error is the equivalent of giving an impossible instruction like, "Draw a square with three sides." If a runtime error isn't addressed, the program will crash and display a traceback. But you can catch runtime errors using `try-except` statements that run error handling code. For example, enter the following into the interactive shell:

```
>>> slices = 8
>>> eaters = 0
>>> print('Each person eats', slices / eaters, 'slices.')
```

This code will display this traceback when you run it:

```
Traceback (most recent call last):
  File "<pyshell#4>", line 1, in <module>
```